RAW: RULES AS WRITTEN

Evaluating LLM & RAG Based TTRPG Rule Assistants with a focus on D&D 5e

Final Project Report

HDIP in Computer Science

Student Name: Leanne Ahern
Student Number: 20108905
Supervisor: Peter Windle

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpts and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student: Leanne Ahern **Date:** 24 March 2026

Preface

This document should be read in conjunction with the following resources:

Trello Board:
https://trello.com/invite/b/653ef5530ad9f40e05841ce8/ATTI58055e25923171e19a60c24901d674838869A0A3/hdip-project

Github Repository: https://github.com/OpinionatedHeron/RAW_DnD_Project

Github Pages Website: http://bit.ly/4lZ4LZt

Table of Contents

Glossary

TTRPG	Tabletop Role Playing Game
D&D	Dungeons and Dragons
5e	Fifth (5th) Edition
SRD	System Reference Document
OGL	Open Game License
WotC	Wizards of the Coast
LLM	Large Language Model
RAG	Retrieval-Augmented Generation

Figures

1. Introduction

The main goal of this investigation and report is to test and analyse different AI models when given a specific ruleset for a game. This particular investigation will focus on the Dungeons and Dragons 5th Edition Basic Rules[1] as it can be difficult to learn due to the level of knowledge and experience required to begin playing. This report aims to determine if an AI model can be used as a rule assistant for Dungeons and Dragons and if so, which model should be recommended based on a criterion of accuracy as compared to effort, usage cost, and latency. It is also to work as a configuration and set up guide for anyone that would like to assess an AI model against a different dataset for their own, unique purposes.

Background

The idea for this project arose from a desire to make tabletop role playing games (TTRPGs) more accessible to new players and improve ease of use for veteran players and game masters. This project was an opportunity to evaluate a number of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) technologies to assess if these tools could be used as a potential Rules Assistant for these kinds of games.

However, for these tools to be considered useful as rule assistants, criteria needed to be investigated and established to prove that the effort would be worthwhile. As I have established knowledge of D&D rules and play requirements, this was a use case that I could investigate thoroughly and assess with confidence.

Objectives

The overall goal of this project is to evaluate a number of different Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) technologies as D&D rules assistants based on their accuracy, financial cost to run or use, latency, and the effort needed to build or use them. This evaluation will be carried out using a gold-standard dataset of questions and answers, validated by a human, based around the Open Gaming License (OGL) System Reference Document (SRD) v5.1 for Dungeons and Dragons Fifth Edition.[2]

[1] SRD-OGL_V5.1, https://media.wizards.com/2016/downloads/DND/SRD-OGL_V5.1.pdf
[2] SRD-OGL_V5.1, https://media.wizards.com/2016/downloads/DND/SRD-OGL_V5.1.pdf

The selected evaluation areas are based around some of the more common pain points and concerns for the average TTRPG player. For a rules assistant to be effective, it needs to be able to correctly and accurately retrieve and interpret rules based on simple, casually phrased questions. If the assistant hallucinates or provides incorrect information, then it could negatively impact the game or someone's experience. The financial cost matters because if players are already investing in materials for the game itself, adding additional cost may not be something they are willing or able to do. If adding a rule assistant also adds significant cost, many players will likely opt not to use such a service. Latency and effort to use are two of the biggest ones in a real-life situation. The goal of a tool like a rule assistant is to make the user's life easier, if it takes too long to load or has a steep learning curve why would they use it, rather than just manually searching the rule books.

For this project, the focus will be on testing LLMs and RAG technology to assess their function and accuracy in the evaluation areas mentioned above. The output should include a set of data that can be used to compare different models across these areas as comprehensively as possible.

In order to test LLMs and RAG technologies, the main goals are as follows:

- Create a gold standard dataset, this will include a variety of beginner friendly questions based on the rules found within the SRD. These will be human validated questions and will be used as a ground truth for all model evaluations.
- Have a clear and detailed guide available within a repository that others can follow if they wish to assess an AI model in a similar manner using their own data.
- Evaluate different models and technologies using the gold standard questions and answers. Assess different artificial intelligence models and technologies, such as NotebookLM, ChatGPT, and Claude. Document areas where models excel and struggle and compare results across specific fields.
- Complete final report with empirical data based on evaluations, and potential recommendations for AI rule assistants based on chosen assessment criteria.

2. Analysis & Initial Research

Before starting this project, there were a few main areas that needed to be established and outlined before committing any work, such as establishing the relevant data and documentation and deciding what technologies are needed to implement this evaluation.

Gold Standard Dataset

The gold standard dataset is the keystone of this evaluation as it creates the standard that the chosen models will be assessed against. In order to begin creating this dataset, the source rule set needed to be chosen. D&D 5e has an abundance of rule books, however, it very quickly became clear that being able to assess such an expansive knowledge base would make validating answers for gold standard questions extremely difficult within the project time period. So the initial rule options were narrowed down to two of the core rulebooks – *The Dungeon Master's Guide* and *The Player's Handbook*. These books are widely considered to be the main entry point for most players. However, as these books are created and distributed by *Wizards of the Coast (WotC),* a subsidiary of *Hasbro,* using them could potentially have some legal and ethical implications regarding copyright.

Through investigating alternative options in order to combat any potential copyright infringement, the System Reference Document version 5.1[3] was most suitable for the purpose of this project. This document is part of the Open Gaming License (OGL) developed by *WotC,* which state that *"in consideration for agreeing to use this License, the Contributors grant You a perpetual, worldwide, royalty-free, non-exclusive license with the exact terms of this License to Use, the Open Game Content"*[4]. This means that the mechanics and rules mentioned in the SRD are available for others to use freely, as long as they are not including any of *WotC's* intellectual property. While there is no specific callout in this document regarding using it within AI, there is a post on their forum, from a third party, stating that it does fit within the terms of their OGL[5]. Version 5.1 was also chosen rather than a newer version because it is also available under a Creative Commons

[3]Wizards of the Coast (2016), SRD-OGL_V5.1, https://media.wizards.com/2016/downloads/DND/SRD-OGL_V5.1.pdf
[4] Wizards of the Coast (2016), SRD-OGL_V5.1, https://media.wizards.com/2016/downloads/DND/SRD-OGL_V5.1.pdf
[5] D&D Beyond, *An AI trained on Dungeons & Dragons*, Post Number 4 (2024) https://www.dndbeyond.com/forums/d-d-beyond-general/general-discussion/214860-an-ai-trained-on-dungeons-dragons?srsltid=AfmBOopJWF2e6tQWKRJbjMbkxvhCCh9z0YJrSiMGajEftkand7hjxkRV

Attribution 4.0 International License ("CC-BY-4.0")[6], ensuring further international use cases. Once this was decided, it provided a solid starting point to begin creating the gold standard dataset which would be used as a benchmark throughout testing.

RAG Implementation

The next step to consider was what technologies should be used to evaluate this dataset once it was created. It needed to be flexible enough to allow a variety of AI models and technologies, in order to complete a more comprehensive investigation. Based on this fact, RAG implementation seemed to be a good place to start. Retrieval-Augmented Generation allows an AI model to access information within external knowledge bases or sources provided to it. It allows a model to potentially find and retrieve more accurate and relevant information for a specific purpose using the provided sources. This is extremely relevant for this investigation as providing a model with the specific rules it is supposed to reference and answer questions on should, in theory, provide improved and more accurate results.

When considering how to utilise a simple RAG implementation, it was decided that NotebookLM would be a good tool to run the initial tests and create some baseline data regarding accuracy and potential weaknesses in the gold standard dataset. NotebookLM is an AI tool that has a free tier to use (with some limitations) so it has an extremely low barrier to entry. Since it is described as being a *"source-grounded"*[7] technology, it should be a simple and easy starting point for testing and evaluation. However, it is important to note that this tool will really only be beneficial in the exploratory phase of this project (specifically testing the potential question bank) as it has limited capabilities when it comes to exporting bulk data.

Model Comparison

This is where the bulk of the testing and investigation will take place, so it was important to consider what models and hosting options to use and test. The main categories considered were large models such as ChatGPT or Claude, and open-source, locally hosted models such as Llama 3, Gemma, and Mistral – both categories are appealing for different reasons.

[6] Wizards of the Coast (2016), System Reference Document 5.1 - CCBY04 License https://www.dndbeyond.com/attachments/39j2li89/SRD5.1-CCBY4.0License.pdf.
[7] Google (2025). NotebookLM. https://notebooklm.google/.

Large proprietary models pull from massive amounts of data and because of this, they tend to be more accurate from the beginning. These models also tend to have more transparent pricing and usage metrics, which would be very beneficial when generating the empirical data for the final report. Methods of testing that were considered at this point were to use these models' internal knowledge base first and assess the accuracy, cost, and latency this way first, and then adding the SRD as additional information to assess if there is a significant difference in accuracy when RAG methodologies are implemented. This testing will require the data to be written in a specific format, such as JSON or CSV to ensure that the questions and results assessment can be done in bulk. This is where a GitHub repo with the necessary environment setup, the dataset, as well as any scripts used to run the chosen models will also need to be created.

The benefit of smaller, open-source models is obviously the lower cost, which is always necessary to consider for any potential real-world application. Since they would be smaller models running locally on a personal device there is more potential to customise and even fine-tune a model with the specific purpose of making it an accurate rule assistant. However, it is important to note that any fine tuning could potentially be out of the scope of this initial project as it is more relevant to building a specific assistant than assessing the functionality of these tools. It is still something that could be considered within this project but would be more of a stretch goal based on the additional training and implementation required. It can also be difficult to obtain accurate metrics of cost and usage from the open-source models in comparison to proprietary models. For these reasons, for at least the initial implementation, a number of proprietary models will be used, with the potential to expand into local, open-source models based on time, progression, and workload.

3. Design

Based on the research carried out and discussed above, the design for this project can be broken down into two main sections: an evaluation infrastructure and the model configurations.

Evaluation Infrastructure

This starts with the gold-standard questions. This is a series of beginner-friendly questions from a variety of different categories from the SRD such as combat, character creation, spells etc. Once the questions are created, the answers need to be validated against the SRD to create an expected answer that the model output can be compared against. Once the dataset is created and validated, it will need to be implemented into a script that can connect to the chosen model to run the questions through and log the results. Once the results are logged, a scoring rubric will be used to assess the output and help to generate the relevant metric information such as latency, cost, and accuracy.

Essentially, the evaluation infrastructure is what is required to actually generate and assess the data that will be evaluated throughout this project.

Model Configurations

As mentioned previously, the two main methods of assessment for the chosen models are a base model configuration and a model + RAG configuration. This allows for an additional assessment category to compare the differences between baseline models versus RAG enhanced models.

A base model will receive the prompt then search its internal knowledge base for an answer, in cases where it may not have accurate information, it can potentially hallucinate and give incorrect responses. In order to achieve this, a method known as context-stuffing will be utilised. This is *"where additional context is injected into a model's input to improve the quality and relevance of its responses."*[8] Basically, a system prompt with the full documentation is injected into the model, and then ideally answers the gold standard questions only using the information in the system prompt.

[8] Kumar, V. (2025), Context Stuffing, Medium, https://medium.com/@vipulkc/context-stuffing-e904b8aa5264.

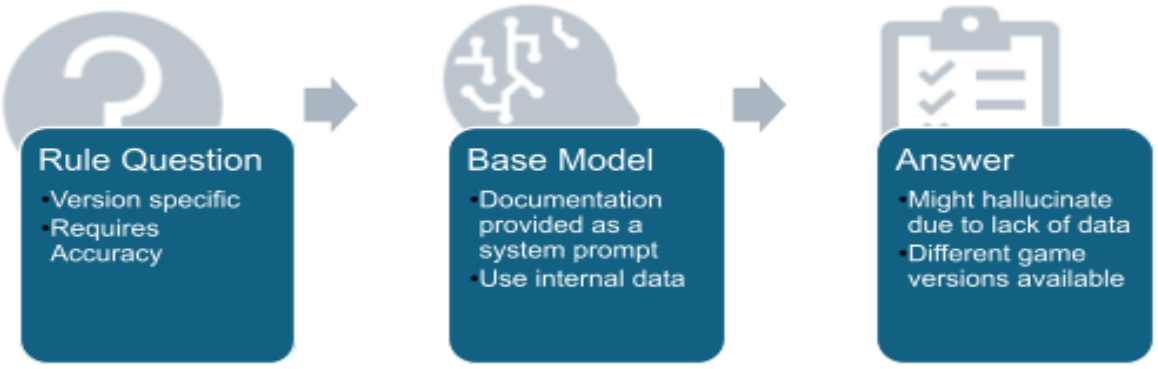

Figure 1: Base Model Configuration

A RAG enhanced model retrieves information from a pre-defined external source, such as the SRD, and pulls information directly from that source, citing where necessary or relevant. It has the potential to be more accurate when utilizing specific documentation such as a ruleset, but this is something that the results should prove or disprove.

To implement this, a RAG pipeline[9] will need to be scripted on top of the existing model. This will require the source document to be loaded into the model through a method known as data indexing. This will involve loading, splitting, embedding, and storing the source document before it can be retrieved by the model.[10]

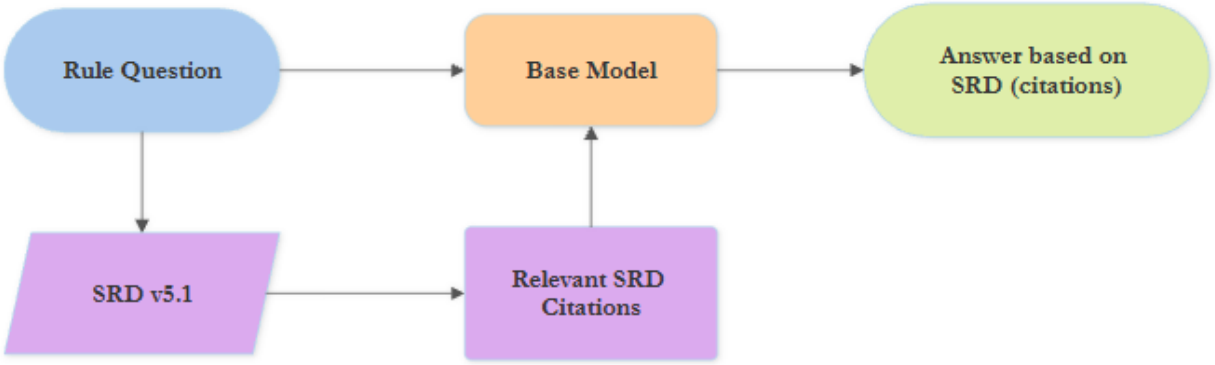

Figure 2: RAG Enhanced Configuration

[9] Idan Novogroder (2024), RAG Pipeline: Example, Tools & How to Build It, lakeFS, https://lakefs.io/blog/what-is-rag-pipeline/

[10] Shin, M. (2025), RAG indexing: Structure and evaluate for grounded LLM answers, Meilisearch.com, https://www.meilisearch.com/blog/rag-indexing

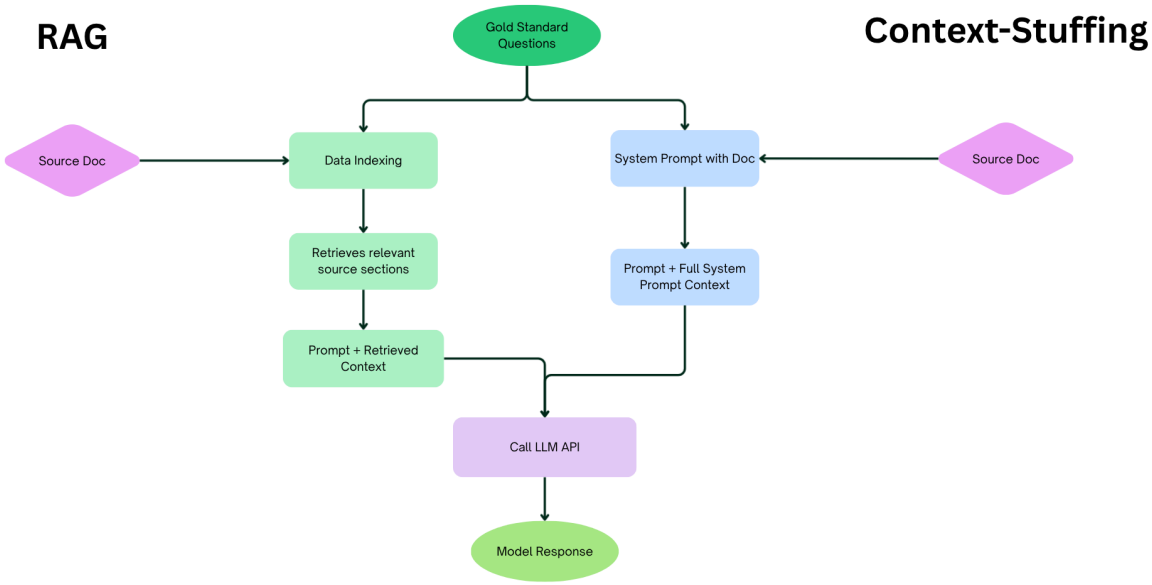

Figure 3: Basic RAG vs Context-Stuffing

4. Methodology

In order to complete this investigation and analysis, a variety of technologies and services will be used – some of which were discussed in the research and analysis section above. Different tools are required for different aspects of this project and are necessary to ensure it is completed to a high standard.

Project Management

For this section, the focus is on tools that will be used to help scope and manage the project as it continues to develop and grow. The main tools used are as follows:

- Git[11] for version control. This will be used during the project implementation phase. Using git in combination with the GitHub project repository[12] will allow for changes to be tracked and documented as the implementation and investigation develops. Github is also an extremely useful resource when working with different packages and libraries, like in this project. It often includes sample code and reference documents that can be used as a source of truth when trying to utilise and understand a technology.
- Trello[13] is used to plan and manage sprints. This provides a visible representation of where the project is currently at, as well as tracking any backlogged tasks and features.

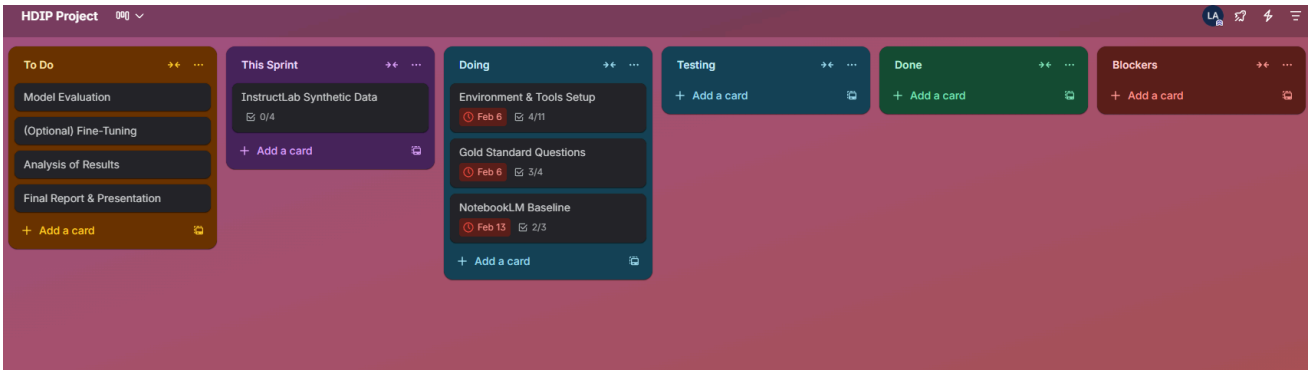

Figure 4: HDIP Project Trello Board

- Perplexity AI[14] was used for research and planning phases. It allowed for quick brainstorming, code and technology investigation start points, and general planning and

[11] Git https://git-scm.com/
[12] Github Repository https://github.com/OpinionatedHeron/RAW_DnD_Project
[13] Trello Board https://trello.com/b/6mCOvmue/hdip-project
[14] Perplexity (2026), Perplexity AI https://www.perplexity.ai/search/i-am-looking-to-investigate-us-CQ7kDYmBTKqNQpyAEefO3Q#5.

scoping of the project. While useful, conversation was used as a guide of where to start investigating, rather than as a source of truth. Due to the fact that Perplexity is described as an *"answer engine"*[15] it is extremely useful in finding and collating relevant sources to complete the evaluation.

Programming Languages

- Python[16] is the main language used throughout this project. It is used in all of the necessary evaluation scripts, as well as the additional document conversion scripts included. This project leverages version 3.11.1[17] within a virtual environment, and should be compatible with later versions if additional package and library versions are also adjusted accordingly.
- JSON is used for all the datasets and results. Specifically, this project makes use of JSONL[18] formats due to the large amounts of data within the implementation of this project. With this, it also uses some CSV and .xlsx file formats in order to make the JSONL files a bit more human readable, if human verification is being implemented.
- The SRD needed to be converted to Markdown[19] in order to be used and understood by the model. While a python script was used to convert the original PDF version of the document to Markdown, it was not formatted correctly which could cause problems when splitting the document in the RAG implementation. So the SRD markdown file had to be manually edited to ensure that everything would work as expected. Also, since one of the goals of this project is to ensure that other developers could use it to evaluate their own models and data, it was important to have a comprehensive and well formatted README to clearly break down the workflow.
- A small amount of YAML was written and is included in the repository after attempting to use InstructLab[20] to create synthetic data. The YAML[21] taxonomy files are not used in

[15] Perplexity (2025), What is Perplexity? | Perplexity Help Center, Perplexity.ai.https://www.perplexity.ai/help-center/en/articles/10352155-what-is-perplexity
[16] Python https://www.python.org/about/gettingstarted/
[17] Python v3.11.1 https://www.python.org/downloads/release/python-3111/
[18] JSONL https://jsonlines.org/
[19] Matt Cone and contributors,. Markdown Guide. https://www.markdownguide.org/
[20] InstructLab, Chen, C., Asghar, J. and Santamaria, L. (2025). Welcome to InstructLab! - docs.instructlab.ai. https://docs.instructlab.ai/
[21] Red Hat (2023). What is YAML? https://www.redhat.com/en/topics/automation/what-is-yaml

the final configuration of the project, but are still included as a guide for any other developers looking to implement InstructLab.

- Bash[22] scripting is also used in this project to install packages and libraries, set up a virtual environment, and run the necessary python scripts. If a developer wishes to use InstructLab, the set up, configuration, and data generation use bash commands to trigger these events. The project repository README also contains all the necessary commands to do this.

AI Models & RAG Technologies

- Different Large Language models are used throughout this project and are evaluated based on the established criteria. The main AI models that are focused on during the initial assessment of the SRD are ChatGPT-5.4[23], Claude Opus 4.6[24], and Gemini 3.1 Pro Preview[25]. It is worth noting that these are all proprietary models, so they do have a cost associated with their token use. All of these models will be evaluated based on the accuracy, cost, and latency criteria previously outlined.

  A stretch goal for this project was to also consider some locally-hosted, open-source models such as Gemma 3[26], Llama 3[27], or Mistral 3[28] as these should work out as a cheaper alternative to proprietary models, and could leave more potential to further expand this project from evaluation to actually creating a specific rule assistant.
- NotebookLM was used for the initial testing and aided in assessing the complexity and validity of the gold standard questions. It establishes some valuable benchmarks for the main dataset, and is a good comparison against the proprietary models. It is possible that NotebookLM functions perfectly as a rule assistant for most gamers as it is affordable and extremely easy to use. The main issue with trying to evaluate this model is that there is no automated method to assess larger amounts of data quickly and easily without a human-in-the-loop.

---

[22] GNU.org (2018). What is Bash? (Bash Reference Manual), https://www.gnu.org/software/bash/manual/html_node/What-is-Bash_003f.html
[23] OpenAI (2025). GPT-5.4 Model | OpenAI API, https://developers.openai.com/api/docs/models/gpt-5.4
[24] Anthropic (2025). What's new in Claude 4.6 Claude API Docs, https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6
[25] Google (2026). Gemini 3.1 Pro Preview Google AI for Developers https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview
[26] Google (2025). Gemma 3 model overview Google AI for Developers https://ai.google.dev/gemma/docs/core
[27] Meta (2022). Llama 3 Meta Llama https://www.llama.com/models/llama-3/
[28] Mistral AI (2025) Introducing Mistral 3 Mistral.ai https://mistral.ai/news/mistral-3

Frameworks & Libraries

- Sentence Transformers[29] is a Hugging Face framework that is used to compute embeddings to be used by a model. This essentially means that it converts text into coordinates so that the model can easily find and access information without having to read the full text, it simply searches for the relevant coordinates. This is part of the RAG pipeline which allows the model to retrieve relevant sections of the source document based on the prompt asked.
- Facebook AI Similarity Search (FAISS)[30] is a library that works with the sentence transformers above. It creates an index of the coordinates created using the sentence transformer framework and then groups them by similarity. For example, melee attacks could include striking with a sword, hitting with a hammer, or swinging an axe; FAISS gathers these together so that if a user asks a question containing similar information, it knows to access those sections of the document.
- *"LangChain is a framework for building agents and LLM-powered applications."*[31] It is an extremely popular framework and is used in many AI implementations in some capacity. However, for this project, text splitter integrations[32] are the only aspect of this framework that is utilised. This library is used to split or 'chunk' the source markdown file. It breaks the file into smaller sections - this can be determined by character count, word count, or markdown headings. These chunks are then what are used when looking for an answer in RAG implementation; rather than looking at the whole document a only a certain number of prompt relevant chunks are read by the model.
- In order to automate the evaluation of results within the project, BERTScore[33], a natural processing language (NLP) evaluation metric was implemented. It measures the semantic similarity between the model output and the golden response and provides a score based on that assessment. This makes it faster and easier to assess and validate larger sets of data for accurate responses.

[29] huggingface (2025). GitHub - huggingface/sentence-transformers: State-of-the-Art Text Embeddings, https://github.com/huggingface/sentence-transformers
[30] Facebook Research (2023). Faiss GitHub https://github.com/facebookresearch/faiss
[31] LangChain (2022). LangChain GitHub https://github.com/langchain-ai/langchain
[32] LangChain Docs (2025) Split markdown - text splitter integration https://docs.langchain.com/oss/python/integrations/splitters/markdown_header_metadata_splitter
[33] Tiiiger (2023). GitHub - Tiiiger/bert_score: BERT score for text generation https://github.com/Tiiiger/bert_score/tree/master

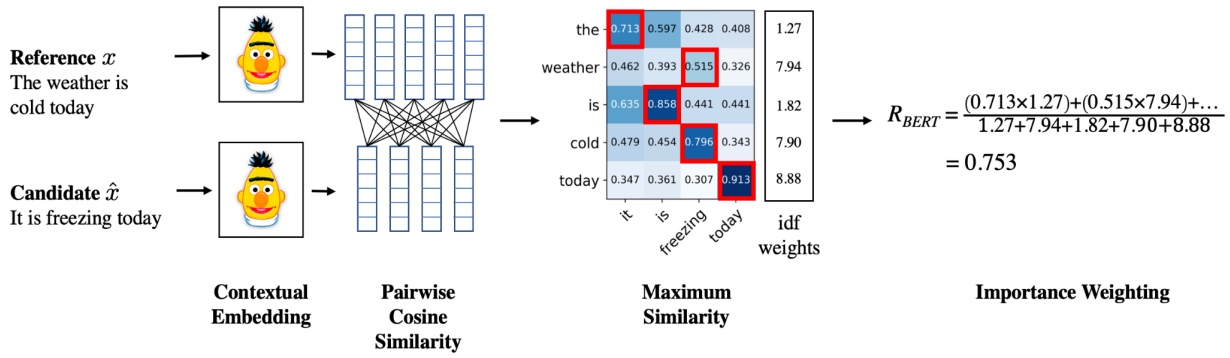

Figure 5: Computing BERTScore[34]

- This project also uses a number of libraries that assist with converting documents from PDF to Markdown, as well as converting JSON files to excel to make them human readable in case an additional manual review is required. A few different PDF conversion libraries are included in the repository because of experimentation with different methods in order to achieve the best result possible. In the end, pymupdf4llm[35] was used in the final implementation as it converted the document with the best formatting (or of the trialled options) without a GPU requirement.
- A number of data and analysis libraries are also used in the final configuration in order to evaluate and visualise the outcomes of the accuracy, latency, and cost tracking. These will be discussed in more detail in the implementation sections of this document as they are much more common libraries that do not require as much explanation as the novel LLM relevant libraries outlined above.

[34] Tiiiger (2025). BERTScore Compute Image GitHub
https://github.com/Tiiiger/bert_score/blob/master/bert_score.png
[35] Artifex Software Inc. (2015). PyMuPDF4LLM - PyMuPDF 1.24.9 documentation Readthedocs.io.
https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/index.html

5. Development

Creating Gold Standard Ruleset

The first step to get this project underway was to create a data set of gold standard questions that would be used as the benchmark comparison against the models. Most of this initial investigation and implementation was completed outside of creating or writing any code. The quality of this evaluation hinged on having a high quality data set that model outputs could be compared to and evaluated against. The data needed to be detailed and complex enough in order to correctly evaluate a model's capabilities to understand and answer difficult edge scenarios that could crop up when real users ask questions; mainly, gold standard questions should not be simple search and retrieval answers, there needed to be more complexity to them.

The process of creating this dataset began with human generated questions. It encompassed the kinds of general questions that the average D&D player would pose in a game scenario, as well as more casual, generic methods of asking rule specific questions, i.e. not using the actual SRD terms, but rather what a new player might understand a condition or concept as. However, it quickly became clear that this was a time consuming method of generation. It also places the burden of quality of these data sets on the creativity of the person writing them. There are scenarios where this could work, for example, having multiple people creating question sets separately, asking them to focus on different areas of the rules when coming up with questions, but that was not a viable option in this case. As a result, 30 human generated questions were created and stored in a Google Sheet[36]. These questions will be used as a baseline to further expand the data set through other means.

Question	Answer	NotebookLM (PDF)		
		Accuracy	Context	Notes
I really want to attack and hit things with my character. Which classes gain an extra attack so I have more chances to hit?	Barbarian, Ranger, Fighter, Monk, Paladin	Correct	Additional Context	Gives a lot of extra information on potential classes and how you can gain an extra attack through features, such as warlock Also mentions multiclassing as an option to consider
As a barbarian, I get a feature called a Brutal Critical at 9th level. What does it mean that I add an additional weapon damage die? What die do I add?	The die depends on the weapon you are using. So if you are using a Greataxe you would add an additional d12 when rolling damage.	Correct	Additional Context	Answer is correct with additional information on the Brutal Critical explaining it in more detail. However, gives the damage die for other weapons also which might be a bit excessive
What bardic inspiration die should I use at level 7?	At level 7, a bardic inspiration dice would be a d8	Correct	Additional Context	
What rolls can Cutting Words be used on?	Attacks rolls, ability checks, damage rolls	Correct	Additional Context	

Figure 6: Golden Questions in Google Sheets

[36] Ahern, L. (2026). Dungeons & Dragons - Gold Standard Questions Google Doc https://docs.google.com/spreadsheets/d/16lMSx3sf811uDQxNP73yeEokRLn6Deyz8_5T84-Zf5E/edit?usp=sharing

Once the questions had been created and the answers validated, they were used to assess NotebookLM. The results from this assessment also provided insight into the potential complexity of these benchmark questions. NotebookLM functions as a RAG based model that utilises the chosen data as a source of truth. It has an extremely simple set up, just create a Notebook and upload or add your chosen source documents, such as the SRD. Adding the SRD did cause some minor issues initially as NotebookLM would not read the full document when added as a Google Doc, even though this is the recommended format. However, this issue was quickly rectified by using the PDF version of the SRD.

One of the main advantages of NotebookLM (outside of its ease of use) is that it only uses the provided source to answer questions. It can still hallucinate or misinterpret questions, but overall it will not pull from additional sources, allowing for a proper assessment of the current dataset. Also, when answering questions it includes links to the section of the source document where it found the answer, and this can be used to help validate or disprove it.

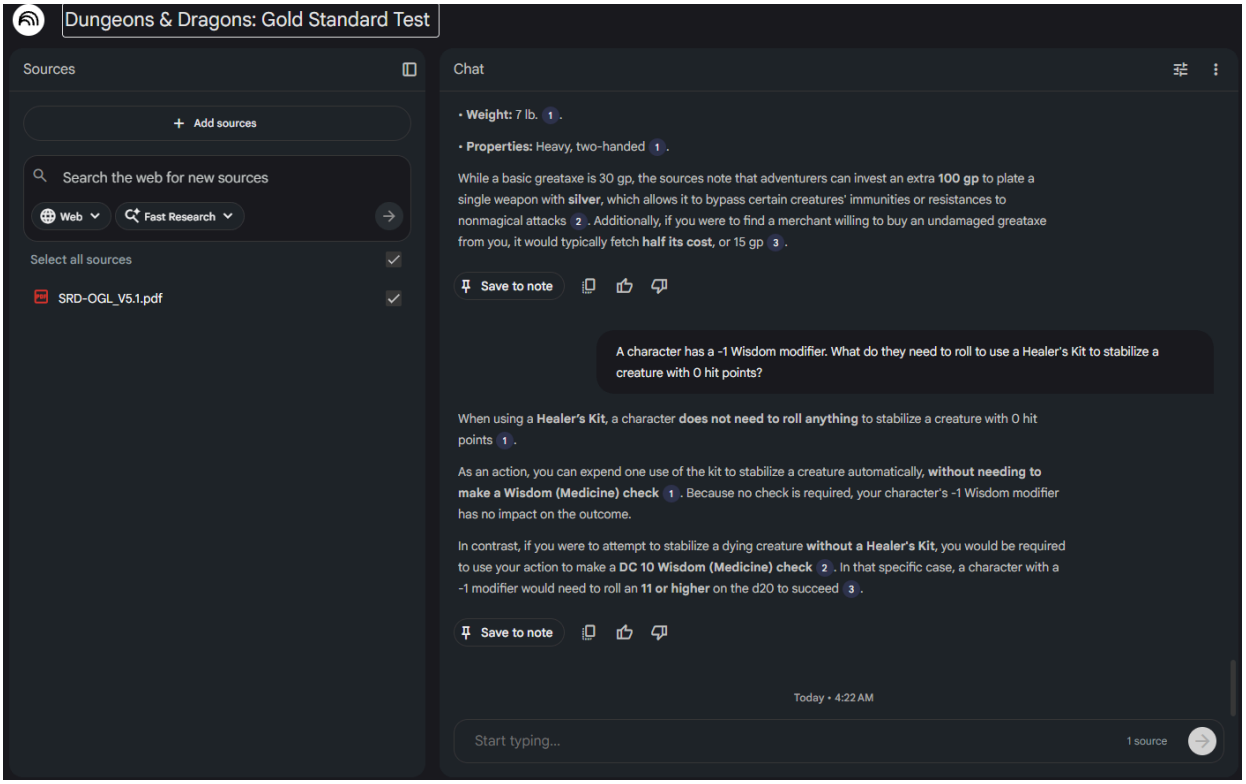

Figure 7: SRD NotebookLM with Questions

Adding the questions into NotebookLM can be a bit slow, because even though it is very accurate, it does have some latency issues. Each prompt takes approximately 15 to 30 seconds to generate a response (not documented or tracked, just simple user observation) which can be a bit

of a wait in a tense game scenario. As can be seen in *Figure 6* included above, when assessing NotebookLM's responses they were graded in two main areas - 1. Accuracy and 2. Context. Accuracy evaluated how correct the response was based on the provided guidelines, and Context assessed how much information was provided. Each response could get a full score of 1 for accuracy, 0.5 for partial accuracy, and 0 for inaccurate or hallucinated information. This scoring system ensured that the overall accuracy of NotebookLM, based on the verified answers, could be calculated with ease. Overall, it scored 98.33% for accuracy with only one question being partially inaccurate due to a slight hallucination as to what counted as a playable race. The high accuracy rate of these responses potentially highlights a lack of complexity within the current data set.

The simplest solution to this issue was to expand the current dataset with the help of AI. This method would involve using the model to generate complex questions that leverage multiple rule sections and then having a human verify the answer to these questions, and that the answer actually exists within the SRD. Claude[37] and Gemini[38] were chosen to further expand the current data set by generating additional questions. A detailed prompt was created for both to use, that emphasised creating complex questions that required an understanding of multiple questions and that only the SRD would be used as a source of truth. These questions were then verified by a human reviewer, who provided a gold standard answer based on the SRD rules. When reviewing these prompts, it was noted that there were a number of questions that included knowledge outside of the specified ruleset such as *"Does standing up from prone provoke opportunity attacks if the creature has the Mobile feat?"*[39]. For this question, the mobile feat does exist in D&D 5e, however, it is only explained in the *Player's Handbook* which is outside the scope of this project. This shows that even when generating prompts models can still feature inaccuracies, in this case, using its internal knowledge base rather than the source provided. This did create the potential for an interesting stretch goal and investigation, where models could be further evaluated by checking if they would hallucinate or generate answers that could not be answered using the SRD, but this was not something that was currently an option due to time constraints and slight scope creep due to the additional time taken on data set creation. The Notebook LM review process was repeated by passing these additional questions into the previously created Notebook. This added 104 additional questions to the initial 30. When assessing these additional questions,

[37] Claude AI (2025) Claude AI Generated Questions Dataset, https://claude.ai/share/ea9f40e3-f597-4b2b-89e9-11df7cd146bb
[38] Google Gemini (2025) Gemini AI Generated Questions Dataset, https://gemini.google.com/share/63011956a2db
[39] Claude AI (2025) Claude AI Generated Questions Dataset, https://claude.ai/share/ea9f40e3-f597-4b2b-89e9-11df7cd146bb

NotebookLM scored 99.52% accuracy, further showcasing the strength of this tool to produce accurate and grounded results, even with a latency issue. If a gamer was looking for an affordable and simple rules assistant, this could be a very valid option. One area that might cause an issue, as seen above, is trying to add larger data sources. If a user wanted to add more rule books for NotebookLM to use as a source of truth, it may not be able to handle or support such large documents even in PDF format.

InstructLab

Based on the NotebookLM results, it appeared as though it would be beneficial to further expand the data set again. A larger data set can offer more comprehensive results as the models are being put under more pressure to perform. However, the current method of generating questions is time consuming and laborious, so an alternative needed to be found. This is where the idea of synthetic data became an appealing option. "*Synthetic data is artificial data designed to mimic real-world data*"[40], it can be used to bolster a given data set in order to gather more results or outcomes. It is generally created using artificial intelligence such as generativeAI. InstructLab[41] is an open-source tool that can help train an LLM and can be used to create synthetic data. The attempt to implement this tool is what caused major scope creep and time management issues within this project.

The first issue encountered was the necessity to convert the SRD PDF to Markdown. After some investigation, there were a number of python scripts that could be used to convert a PDF file. The most simplified version was fitz[42] a library imported from pymupdf. This worked quickly and was extremely easy to script. However, the converted document was not formatted at all and was incredibly difficult to read. Then marker-pdf was tried, however this had a GPU requirement, and when trying to run the script the process kept being killed. Finally, pymupdf4llm provided a semi-usable output. The markdown file still had a number of issues that needed to be fixed manually, such as broken and poorly formatted tables and some incorrect spacing and heading issues.

[40] Caballar, R.D. (2023). What is synthetic data? Ibm.com, https://www.ibm.com/think/topics/synthetic-data
[41] Red Hat (2025). What is InstructLab? Redhat.com https://www.redhat.com/en/topics/ai/what-is-instructlab
[42] Python Coding (2024). PDF Manipulation using Python — fitz Library, Medium https://pythonclcoding.medium.com/pdf-manipulation-using-python-fitz-library-619227d59f3d

```
## **Elemental Gem**

_Wondrous item, uncommon_

This gem contains a mote of elemental energy. When you use an action to break the gem, an elemental is summoned as if you had cast the _conjure elemental_ spell, and the gem's magic is lost. The type of gem determines the elemental summoned by the spell.

|**Gem**<br>|**Summoned Elemental**<br>|| |---|---|---| |Blue sapphire<br>|Air elemental<br>|| |Yellow diamond<br>|Earth elemental<br>|| |Red corundum|Fire elemental|| |Emerald|Water elemental||
```

Figure 8: Broken Markdown Table - Elemental

```
16055   #### **Wand of Wonder**
16057   _Wand, rare (requires attunement by a spellcaster)_
16058
16059   This wand has 7 charges. While holding it, you can use an action to expend 1 of its charges and choose a target within 120 feet
        of you. The target can be a creature, an object, or a point in space. Roll d100 and consult the following table to discover what
        happens.
16060
16061   If the effect causes you to cast a spell from the wand, the spell's save DC is 15. If the spell normally has a range expressed in
        feet, its range becomes 120 feet if it isn't already.       You, 4 seconds ago • Uncommitted changes
16062
16063   If an effect covers an area, you must center the spell on and include the target. If an effect has multiple possible subjects,
        the GM randomly determines which ones are affected.
16064
16065   The wand regains 1d6 + 1 expended charges daily at dawn. If you expend the wand's last charge, roll a d20. On a 1, the wand
        crumbles into dust and is destroyed.
16066
16067   |**d100**|**Effect**|| |---|---|---| |01-05|You cast slow.|| |06-10<br>11-15|You cast faerie fire.<br>You
        are stunned until the start of your next turn,<br>|| ||believing something awesome just
        happened.|| |16-20|You cast gust of wind.|| |21-25|You cast detect thoughts on the target
        you<br>|| ||chose. If you didn't target a creature, you<br>|| ||instead take 1d6 psychic
        damage.|| |26-30|You cast stinking cloud.|| |31-33|Heavy rain falls in a 60-foot radius centered
        on<br>|| ||the target. The area becomes lightly obscured.<br>|| ||The rain falls until the
        start of your next turn.|| |34-36|An animal appears in the unoccupied space|| ||nearest the
        target. The animal isn't under your|| ||control and acts as it normally would. Roll a
        d100|| ||to determine which animal appears. On a 01-25,<br>a rhinoceros appears; on a
        26-50, an elephant|| ||appears; and on a 51-100, a rat appears.|| |37-46|You cast lightning
        bolt.|| |47-49|A cloud of 600 oversized butterflies fills a 30-foot<br>radius centered on the
        target. The area becomes|| ||heavily obscured. The|butterflies remain for 10| ||minutes.|| |50-53|You
        enlarge the target as if you had cast<br>|| ||enlarge/reduce. If the<br>|target can't be
        affected by<br>| ||that spell, or if you didn't target a creature, you<br>|| ||become the
        target.|| |54-58|You cast darkness.|| |59-62|Grass grows on the ground in a 60-foot radius<br>|| ||
        centered on the target. If grass is already there,<br>|| ||it grows to ten times its
        normal size and remains|| ||overgrown for 1 minute.|| |63-65|An object of the GM's choice
        disappears into the<br>Ethereal Plane. The object must be neither worn|| ||nor carried,
        within 120 feet of the target, and no|| ||larger than 10 feet in any dimension.|| |66-69|
        You shrink yourself as if you had cast<br>|| |70-79|enlarge/reduce on yourself.<br>You cast
        fireball.|| |80-84|You cast invisibility on|yourself.| |85-87|Leaves grow from the target. If you
        chose a point|| ||in space as the target, leaves sprout from the<br>creature nearest to
        that point. Unless they are|| ||picked off, the leaves turn brown and fall off|| ||after
        24 hours.|| |88-90|A stream of 1d4 x 10 gems, each worth 1 gp,<br>|| ||shoots from the
        wand's tip in a line 30 feet long<br>|| ||and 5 feet wide. Each gem deals 1
        bludgeoning<br>|| ||damage, and the total|damage of the gems is|
16068
```

Figure 9: Broken Markdown Table - Wand Effects

```
#### **Elemental Gem**

_Wondrous item, uncommon_

This gem contains a mote of elemental energy. When you use an action to break the gem, an elemental is summoned as if you had cast the _conjure elemental_ spell, and the gem's magic is lost. The type of gem determines the elemental summoned by the spell.

| Gem | Summoned Elemental |
|---|---|
| Blue sapphire | Air elemental |
| Yellow diamond | Earth elemental |
| Red corundum | Fire elemental |
| Emerald | Water elemental |
```

Figure 10: Fixed Markdown Table - Elemental

While this was not an overly difficult task, it was far more time consuming than originally planned. After a few of the tables were manually fixed, Claude Code was able to automatically evaluate and fix the remainder, which did save a significant amount of time. However, there was no way to fully automate the spacing and heading issues. The benefit to taking the time to do this, was that this file was later needed for the RAG pipeline in a Markdown format, so even if InstructLab did not function as expected, this process was still necessary for the overall LLM evaluation.

Once the source document was formatted, InstructLab was installed and configured.

Install & Configure

```
# Install InstructLab
pip install instructlab

# Verify installation
ilab --version

# Initialise ilab
ilab config init

# Download Teacher model - defaults to merlinite-7b-lab-GGUF
ilab model download
```

Figure 11: InstructLab Commands

InstructLab defaults to a local model, in this case merlinite-7b-lab-GGUF. This is the teacher model that reviews the created seed data and uses that as a baseline to generate the synthetic data. The device running InstructLab has an Nvidia GeForce RTX 2060 graphics card and it was assumed that this would be enough power to run the local version.

Before generating the data, the InstructLab Taxonomy must be created. This was another time consuming task. InstructLab needs a very specific knowledge set in order to create synthetic data. This knowledge set is to be stored in the InstructLab taxonomy directory, and has a very strict format to be used.

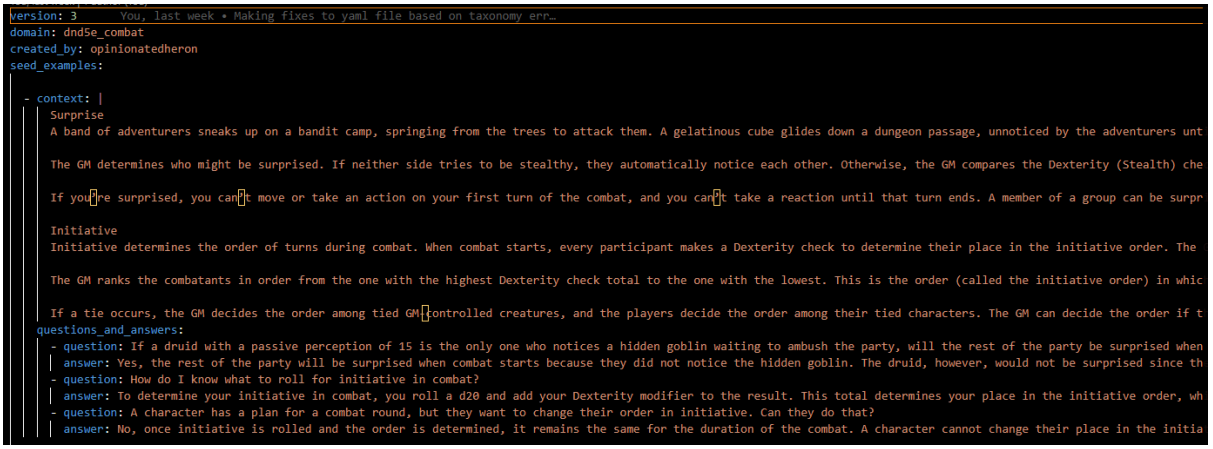

Figure 12: InstructLab Taxonomy Format

Essentially, the taxonomy requires a qna.yaml file that contains examples of the kinds of questions and answers expected for your chosen topic that will then be used to generate the synthetic data. The issue is that these kinds of questions require context to pull from, such as a paragraph from the source document. So in this case, entirely new questions had to be written for the seed data because the initial gold standard questions could not be used as they intentionally crossed different rules and sections to be more complex. Also, InstructLab requires a minimum of five examples in the qna.yaml file, with at least 3 questions under each context section. It also prefers information to be categorised by type. So, in the case of the D&D rules, having a variety of different contexts in one file can be complicated for the model to parse and understand, so separating them by specific categories is preferred. Again, this was another simple enough process that just took time due to the nature of the file structure.

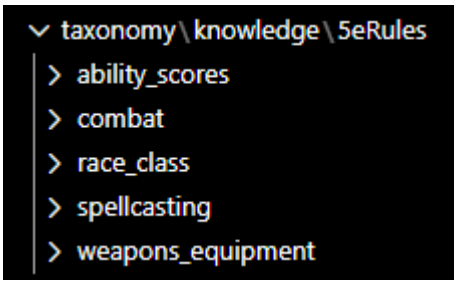

Figure 13: Taxonomy File Structure

Once the taxonomy is created, it gets checked to see if it is actually formatted correctly. If not, it will not allow the user to proceed with the data generation. The benefit is that it does clearly explain what areas need to be fixed within the taxonomy.

```
(dnd-project) opinionatedheron@LAPTOP-SEMPBVS0:~/.local/share/instructlab/taxonomy$ ilab taxonomy diff
knowledge/5eRules/ability_scores/qna.yaml
knowledge/5eRules/spellcasting/qna.yaml
knowledge/5eRules/race_class/qna.yaml
knowledge/5eRules/combat/qna.yaml
knowledge/5eRules/weapons_equipment/qna.yaml
ERROR 2026-03-22 02:28:47,935 instructlab.schema.taxonomy:134: knowledge/5eRules/ability_scores/qna.yaml:117:466 trailing spaces (trai
ing-spaces)
ERROR 2026-03-22 02:28:47,935 instructlab.schema.taxonomy:134: knowledge/5eRules/ability_scores/qna.yaml:136:23 no new line character a
t the end of file (new-line-at-end-of-file)
ERROR 2026-03-22 02:28:51,288 instructlab.schema.taxonomy:134: knowledge/5eRules/race_class/qna.yaml:123:78 trailing spaces (trailing-s
paces)
ERROR 2026-03-22 02:28:51,288 instructlab.schema.taxonomy:134: knowledge/5eRules/race_class/qna.yaml:169:23 no new line character at th
e end of file (new-line-at-end-of-file)
ERROR 2026-03-22 02:28:51,413 instructlab.schema.taxonomy:134: knowledge/5eRules/combat/qna.yaml:82:159 trailing spaces (trailing-space
s)
ERROR 2026-03-22 02:28:51,529 instructlab.schema.taxonomy:134: knowledge/5eRules/weapons_equipment/qna.yaml:138:23 no new line characte
r at the end of file (new-line-at-end-of-file)
Reading taxonomy failed with the following error: 6 total errors found across 5 taxonomy files!]
Traceback (most recent call last):
  File "/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/instructlab/taxonomy/diff.py", line 88, in diff
    validate_taxonomy(taxonomy_path, taxonomy_base, yaml_rules)
  File "/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/instructlab/utils.py", line 465, in validate_taxonomy
    raise TaxonomyReadingException(
instructlab.schema.taxonomy.TaxonomyReadingException: 6 total errors found across 5 taxonomy files!
```

Figure 14: Taxonomy Failure

```
(dnd-project) opinionatedheron@LAPTOP-SEMPBVS0:~/.local/share/instructlab/taxonomy/knowledge$ ilab taxonomy diff
knowledge/5eRules/combat/qna.yaml
knowledge/5eRules/weapons_equipment/qna.yaml
knowledge/5eRules/race_class/qna.yaml
knowledge/5eRules/ability_scores/qna.yaml
knowledge/5eRules/spellcasting/qna.yaml
Taxonomy in /home/opinionatedheron/.local/share/instructlab/taxonomy is valid :)
(dnd-project) opinionatedheron@LAPTOP-SEMPBVS0:~/.local/share/instructlab/taxonomy/knowledge$
```

Figure 15: Valid Taxonomy

Once the taxonomy is marked valid, data can be generated. This is where the main time restraint issue became apparent. The commands to generate synthetic data are as follows:

Generate Data

```
# Generate full pipeline - high quality, more in-depth
ilab data generate --pipeline full --sdg-scale-factor 10

# Generate simple pipeline - lower quality, much faster
ilab data generate --pipeline full --sdg-scale-factor 10
```

Figure 16: InstructLab - Generate Data

Pipeline full generates higher quality data, but takes more time as it consumes much more RAM. Also sdg-scale-factor sets the number of synthetic examples created for each seed example. ilab data generate --pipeline full --sdg-scale-factor 20 is the command that was used in the first attempt to generate data, it should have created about 100 questions based on the seed examples created. However, it was clear very early that something was not working correctly. The data was generated incredibly slowly, about 30 seconds per iteration with over 11,000 iterations to go

through (which would take about 92 hours). After some investigation, it appeared that InstructLab was only using the device CPU and not the GPU. The running command was cancelled and the GPU configuration was updated within InstructLab and the WSL2 environment.

```
cat > /mnt/c/Users/le_am/.wslconfig << 'EOF'
[wsl2]
memory=12GB
processors=4
swap=8GB
EOF
wsl --shutdown
sudo apt install wsl
wsl --shutdown
```

Figure 17: WSL Configuration

Once the configuration was accepted, the generate command was run again with no changes. This time the generation appeared to be going well, the speed had decreased to approximately 7 seconds per iteration, which would still take about 21 hours to run, but it was a significant improvement. This generation process did cause noticeable lag on the device, which made it difficult to work on other aspects while it was running. The process was left alone to run, and after several hours running it failed without generating any data with a connection error.

```
gen_spellcheck Prompt Generation: 100%|##########| 11335/11335 [22:23:59<00:00,  7.11s/it]  2~^[[2~^[[2~^[
INFO 2026-03-24 02:12:14,395 instructlab.sdg.pipeline:199: Running block: flatten_auxiliary_columns
INFO 2026-03-24 02:12:14,419 instructlab.sdg.pipeline:203: Batching disabled; processing block 'flatten_auxiliary_columns' single-threaded.
INFO 2026-03-24 02:12:23,417 instructlab.sdg.pipeline:199: Running block: rename_to_document_column
INFO 2026-03-24 02:12:23,422 instructlab.sdg.pipeline:203: Batching disabled; processing block 'rename_to_document_column' single-threaded.
INFO 2026-03-24 02:12:23,581 instructlab.sdg.pipeline:199: Running block: gen_knowledge
INFO 2026-03-24 02:12:23,581 instructlab.sdg.pipeline:203: Batching disabled; processing block 'gen_knowledge' single-threaded.
gen_knowledge Prompt Generation:  10%|9         | 2185/22670 [19:08:59<248:13:10, 43.62s/it]/tmp/pip-install-hjrcimyl/llama-cpp-python_a584186803634cd0ab9a84859dd21f42/vendor/llama.cpp/ggml/src/ggml-cuda/ggml-cuda.cu:70: CUDA error
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libggml-base.so(+0x1593b)[0x7b77daa8e93b]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libggml-base.so(ggml_abort+0x156)[0x7b77daa8ecd6]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libggml-cuda.so(+0x6ac06)[0x7b77d606ac06]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libggml-cuda.so(+0x6c193)[0x7b77d606c193]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libggml-base.so(ggml_backend_sched_synchronize+0x2e)[0x7b77daaa18ce]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libllama.so(llama_synchronize+0x14)[0x7b77dac08204]
/home/opinionatedheron/venvs/dnd-project/lib/python3.11/site-packages/llama_cpp/lib/libllama.so(llama_get_logits+0xd)[0x7b77dac082dd]
/lib/x86_64-linux-gnu/libffi.so.8(+0x7e2e)[0x7b77dc8b7e2e]
/lib/x86_64-linux-gnu/libffi.so.8(+0x4493)[0x7b77dc8b4493]
/usr/lib/python3.11/lib-dynload/_ctypes.cpython-311-x86_64-linux-gnu.so(+0xa99d)[0x7b77dc8d199d]
/usr/lib/python3.11/lib-dynload/_ctypes.cpython-311-x86_64-linux-gnu.so(+0x13da2)[0x7b77dc8dada2]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyObject_MakeTpCall+0x22c)[0x4e75dc]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyEval_EvalFrameDefault+0x8f2)[0x4fb152]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x5595a3]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyEval_EvalFrameDefault+0x6c6)[0x4faf26]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x5d4efd]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyEval_EvalFrameDefault+0x1b02)[0x4fc362]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyFunction_Vectorcall+0x173)[0x531823]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyObject_FastCallDictTstate+0x2c4)[0x4ef0c4]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyObject_Call_Prepend+0x59)[0x53c709]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x5cbea7]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(PyObject_Call+0x1c5)[0x5412a5]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x5bc60c]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyObject_MakeTpCall+0x22c)[0x4e75dc]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x6bab3b]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x50d977]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11(_PyEval_EvalFrameDefault+0x4781)[0x4fefe1]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x55e097]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x55c888]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x67a774]
/home/opinionatedheron/venvs/dnd-project/bin/python3.11[0x653ae8]
/lib/x86_64-linux-gnu/libc.so.6(+0x94ac3)[0x7b77dd094ac3]
/lib/x86_64-linux-gnu/libc.so.6(+0x1268d0)[0x7b77dd1268d0]
failed to generate data with exception: PipelineBlockError(<class 'instructlab.sdg.blocks.llmblock.LLMBlock'>/gen_knowledge): Connection error.
gen_knowledge Prompt Generation:  10%|9         | 2185/22670 [19:11:21<179:54:14, 31.62s/it]
```

Figure 18: InstructLab Failure

After this failure, a further investigation was attempted. The process appeared to be crashing because of some CUDA GPU error. Reviewing the istructlab/config.yaml, the gpu_layers were adjusted to 28 as a potential fix. The default for gpu_layers in -1 which keeps everything running on the GPU which can put it under pressure, adjusting it to 28 keeps 28 layers on the GPU and pushes the rest to CPU, hopefully alleviating the burden on the GPU in order to complete the test.

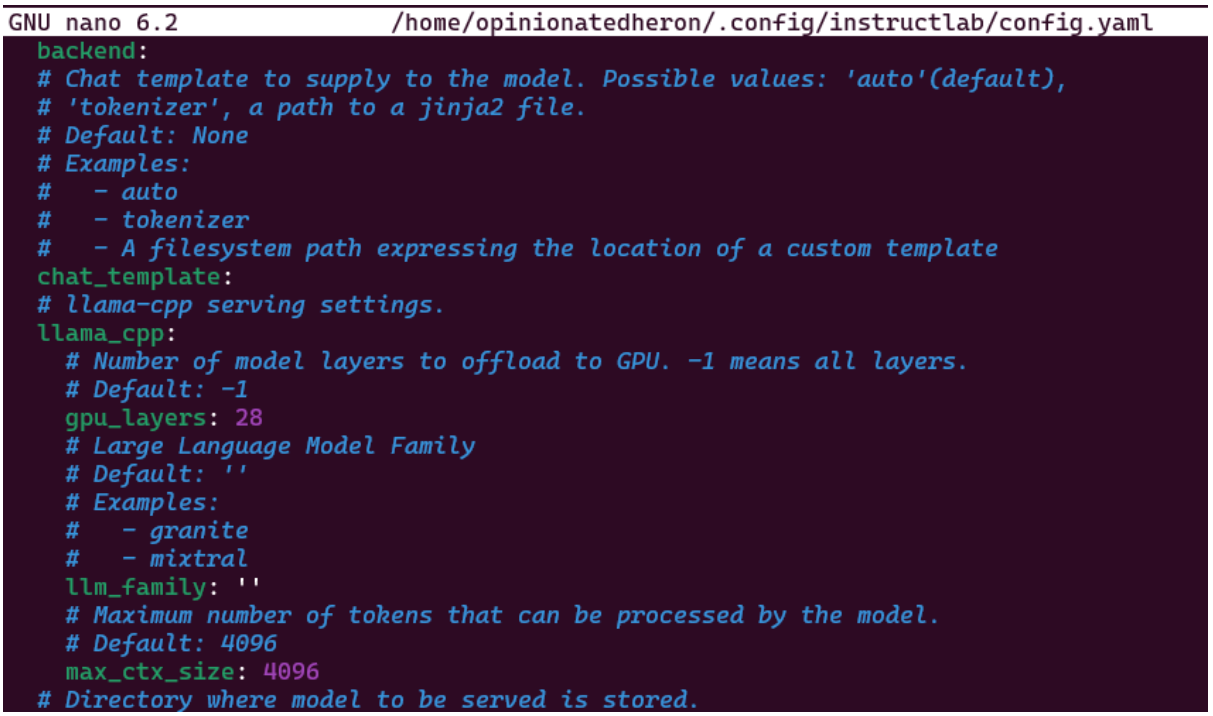

Figure 19: InstructLab config.yaml - gpu_layers

This time, due to how long it takes for this process to run and potentially fail, a simple pipeline was run instead. This would be the final time that this process was attempted, the time lost for little to no gain was not worth remaining stuck on it. The goal of adding InstructLab synthetic data was to increase the data set and have the opportunity to use and work with new, novel software. However, even after the troubleshooting and even reducing the scale to 5, the simple pipeline generation process also failed.

Figure 20: InstructLab generate - simple pipeline failure

If there were no time constraints with this project, an option that could be tested here would be to use a model API rather than a locally hosted model, which would bypass any RAM or GPU issues. However, with the time already lost to this endeavour it was not worth further pursuit at present. Being critical of the overall project process, this should have been considered as a stretch goal to implement after having at least some initial evaluations completed, rather than in the middle of the project.

Model Evaluation

This is the main body of the project, where ChatGPT, Claude, and Gemini are put to the test against the gold standard data set that has been previously defined and validated. Compared to trying to implement InstructLab, this section was pretty standard to implement with no major errors throughout. The initial 134 gold standard questions were broken down into 4 different random JSONL files each with approximately 33 questions. This was done in order to control the evaluation, allowing a smaller starter evaluation that could be expanded on if the implementation works smoothly. The original gold standard google sheet was tidied up with the necessary headings and converted to a JSONL using a simple python script.

q_id	gold_question	gold_answer	origin
37	If a spell requires verbal components and the caster is underwater without magical means to breathe/speak, can the spell be cast?	No, they cannot cast the spell because they cannot make the specific sounds required under water	claude_gen
52	If a creature is both paralyzed and petrified, which condition's rules take precedence for being attacked?	Both rules apply at the same time, neither take precedence both paralyzed and petrified conditions occur simultaneously	claude_gen
80	Does a +1 weapon overcome resistance to non-magical damage?	Yes, a +1 weapon is magical for the purpose of overcoming resistances	claude_gen
62	If a creature is subjected to two effects that each require the same saving throw on the same turn, do they make one save or two?	They need to make 2 saving throws as they are still 2 separate effects even if they are the same saving throw skill	claude_gen

Figure 21: Formatted Gold Standard Questions

Once the gold standard questions were available in the project repository as JSONL files, it was time to create the model wrapper. This script calls the models in the same way, and can be reused in other scripts throughout the project. This is where the latency and cost metrics are defined. The cost is pulled directly from the model sites and calculated based on those figures. The latency is calculated by minusing the start time from the end time and multiplying it by a 1000 to get the number of milliseconds it takes to form an answer. One of the only frustrations with this script were the model software developer kits (SDKs). Overall the chosen models have very similar structures and naming conventions, but there are one or two sections where it is

important to double check what format that model uses. For example, when returning the answer ChatPGT uses response.choices[0].message.content, Claude uses response.content[0].text, and Gemini uses response.text, all are incredibly similar but using the incorrect value would cause the script to throw an error when run. Also, the current version of this script has Gemini commented out, this was because of an issue that occurred during the context stuffing evaluation.

After the model wrappers were created, the context-stuffing evaluation was created. This evaluation method functions by creating a system prompt with the full documentation included, then the gold standard questions are passed into the model after the system prompt. The script then creates a results file that documents the model outputs for the chosen criteria, this is saved as an appended results JSONL (appended just because if the script had to be run multiple times, it doesn't overwrite the first run, just adds to it). It is running this script that might catch some errors from the wrapper file as the models are called from that script. The first run caught an issue with the value for max tokens that needed to be adjusted.

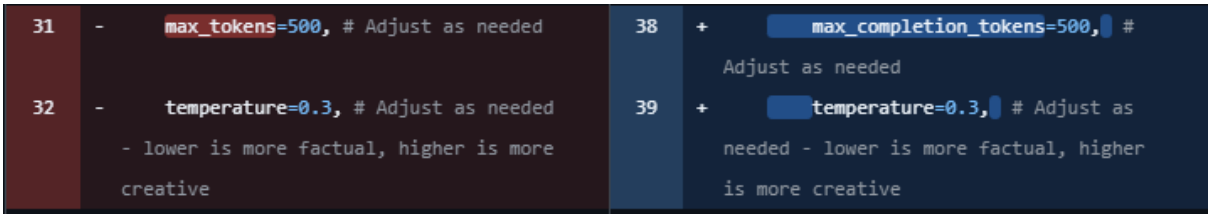

Figure 22: Max Tokens Fix

With the max tokens value adjusted, the script ran again. Claude and ChatGPT ran with no problems, however, Gemini ran the first 5 questions before flagging an error -

"error": "429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \\n Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_paid_tier_input_token_count, limit: 2000000, model: gemini-3.1-pro*

This indicates that the context stuffing used all the available tokens in the Gemini API plan with just 5 questions. With a bit of investigation, this appears to be an issue with the fact that Gemini uses a Tokens per Minute (TPM) metric, so when the context stuffing script tries to add all the prompts to the model, including the system prompt, in one go it exceeds the TPM metric of 2

million. This might be a simple fix by adding a slight delay between each prompt, but there was no time to investigate further. This is something that could be looked into another time.

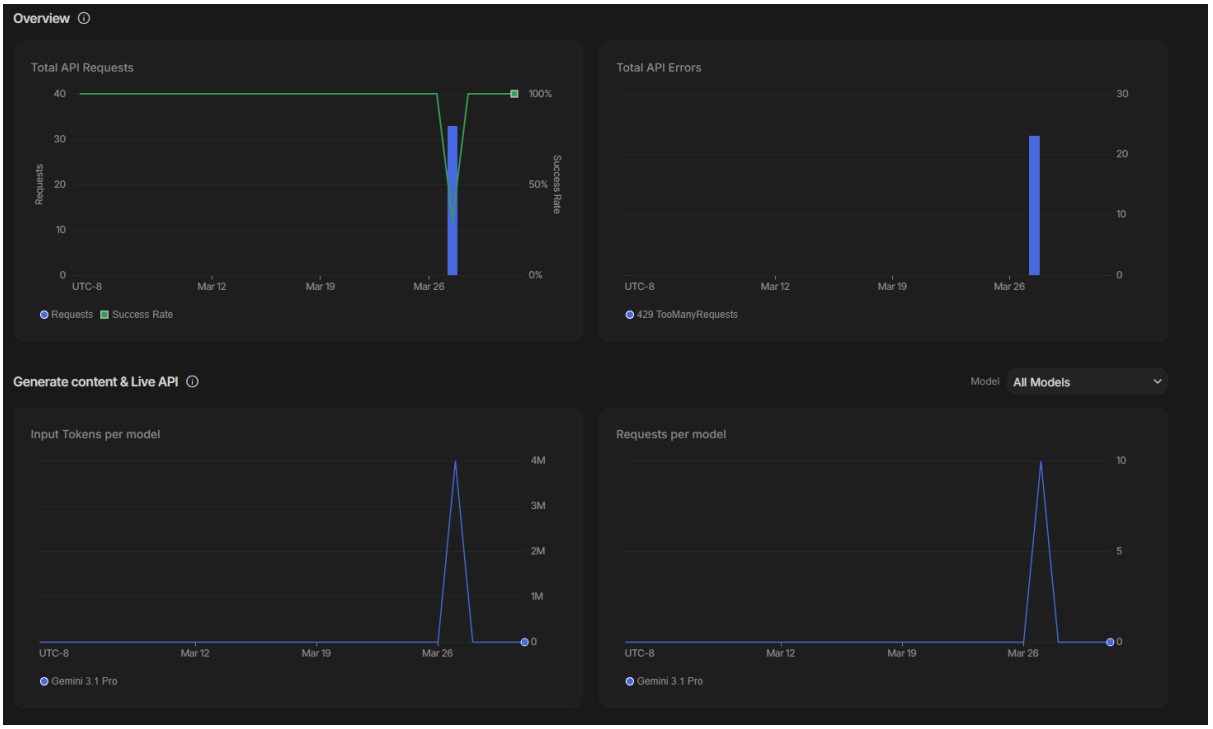

Figure 23: Google AI Studio - Usage Dashboard

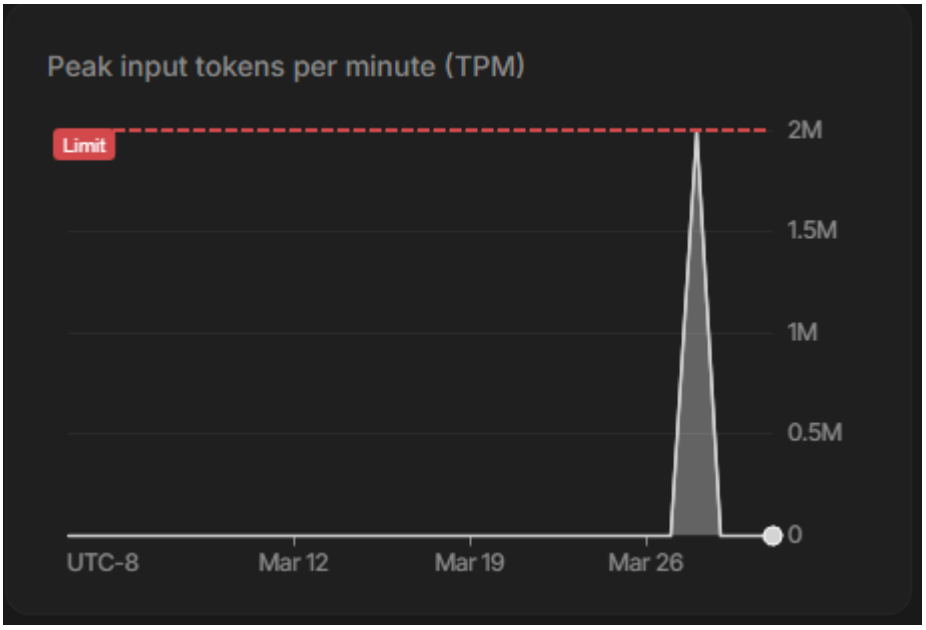

Figure 24: Google AI Studio - TPM

That is all that was needed for the context stuffing evaluation. From here, it was time to move onto the RAG implementation, which was a little more in depth. It started with the creation of the RAG pipeline, which mostly deals with the data indexing and retrieval sections. In this script,

the SRD markdown file is broken into chunks divided by the document headers and the word count. Once the chunks are created, they are then built into an index using sentence transformers and FAISS. The sentence transform embeds the chunk (assigns a coordinate) and adds each chunk to an array. FAISS takes those chunks and measures the vectors in order to store like with like. So then, when a prompt is fed to the model, rather than searching through the entire document, it only uses the top 5 chunks that are related to the question being asked. When this script is first written and run, the index will need to be built which can take a few minutes, after that it will only rebuild the index if the source document changes in any way.

From here, all that remains is the RAG evaluation. This script is very similar to the context stuffing script, just that the system prompt does not need to include the SRD, and a build rag function is added to retrieve the relevant chunks.

Now that all the evaluation scripts had been run, it was time to look at the results. As this project only used 33 gold standard questions in the end, it could certainly just be assessed through human evaluation. However, it was important to consider scaling and the likelihood of wanting to use larger datasets, so automated scoring scripts were written.

As previously discussed, the first method used to assess and score AI accuracy is BERTScore. It looks at the semantic similarities between the model response and the gold standard response and assigns a value based on how similar they are, anything above 0.70 is considered a good score, and anything below is not great.

The other scoring method used, was LLM-As-Judge[43]. This method uses an LLM, usually a smaller model, to assess the accuracy of the model answers when compared to the gold standard response and score it against a provided rubric. In this case, GPT-4o-mini was the chosen model as it is much faster and more affordable to run and the provided rubric was to score the model response between 0 and 2. These 2 evaluation methods are included in the same script, and the results are saved in a single JSONL file.

From here, it was a matter of visualising and curating the data. This was done with one final python script that utilises the matplotlib, numpy, and pandas libraries to assist in aggregating the necessary data and presenting in a comprehensible and human readable manner. The matplotlib library was the most useful as it allows the data to be displayed in a chart, simply create the script for the kind of chart needed, with the relevant data, then once the script is run, the chart is saved

[43] Ip, J. (2025). Leveraging LLM-as-a-Judge for Automated and Scalable Evaluation Confident-ai.com https://www.confident-ai.com/blog/why-llm-as-a-judge-is-the-best-llm-evaluation-method

to the chosen directory. There can be a little of adjusting here, depending on the chosen graph and the relevant data titles as they can sometimes look a bit cluttered.

Model	Test	Avg Latency (ms)	Avg BERTScore	Avg Judge Score	Total Cost	Count
Claude Opus 4.6	Context Stuffing	55730.38	0.84	1.79	72.06	33
Claude Opus 4.6	RAG	7908.68	0.83	1.61	00.40	33
Gemini 3.1 Pro Preview	Context Stuffing	9870.53	0.85	1.2	15.99	33
GPT-5.4	Context Stuffing	57573.79	0.85	1.79	31.81	33
GPT-5.4	RAG	2371.18	0.84	1.3	0.12	33

Figure 25: Results 1 - Table

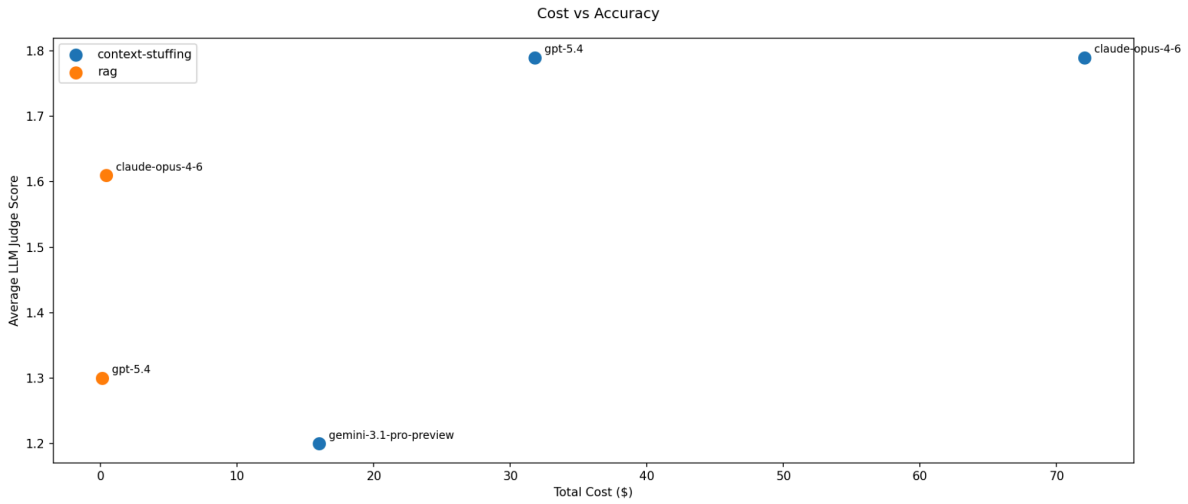

Figure 26: Results 2 - Scatter Graph

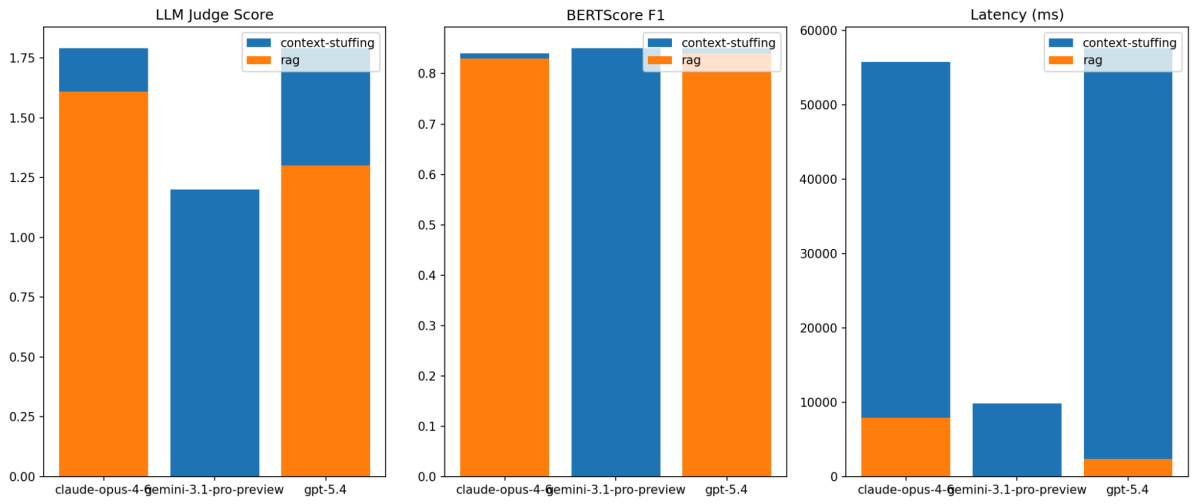

Figure 27: Results 3 - Bar Chart

6. Reflection

What I Learned?

This project was much more in depth than I had initially considered and scoped. It was great to be able to experiment and learn about a new, novel technology and technology stack that was not deeply considered throughout the course. Being able to leverage the skills I have developed over the last 2 years, such as basic python, git version management, even little things like markdown, to investigate a topic that I would not have been able to comprehend previously.

I have learned so much about Large Language Models and their capabilities, but also some of their weaknesses and grey areas. With this evaluation for example, the expectation based on NotebookLM's accuracy levels and the general consensus around the value of RAG implementation, I expected those models to perform better when it came to accuracy. However, looking at the charts and general results, context-stuffing scored significantly higher on both models. Now, this could lead to further investigation, such as maybe my defined section chunks were too small or large and that effected what context could be pulled, or maybe I needed to pull a greater number of chunks to provide the model with better context.

One area I can definitely see where I need to improve and further develop my skills is in time management. There are a few small issues, for example, I think the context stuffing would work out cheaper if I added the caching method that was included in RAG pipeline script, but because I lost so much time working on InstructLab, I didn't want to waste more time trying to implement quality of life improvements when the script ran as I needed it to.

Also, I was surprised by the cost that went into evaluating these models. I had initially budgeted €50 per model, as I thought this would provide a decent baseline. However, as I ran the 33 gold standard questions through the context stuffing implementation, it was shocking to see how quickly tokens were used up.

Overall, this was a valuable learning experience, but it was definitely filled with some harsh realities along the way. While I am delighted with the work I did achieve with this project, I can't help but also see where I need to continue to develop my skills, experience, and knowledge.

Bibliography

Reference list

1. Python Software Foundation (2001). *Functools — Higher-order Functions and Operations on Callable Objects*. [online] Python Documentation. Available at: https://docs.python.org/3/library/functools.html.
2. Abonia Sojasingarayar (2024). *BERTScore Explained in 5 minutes*. [online] Medium. Available at: https://medium.com/@abonia/bertscore-explained-in-5-minutes-0b98553bfb71.
3. Ahern, L. (2026). *Dungeons & Dragons - Gold Standard Questions*. [online] Google Docs. Available at: https://docs.google.com/spreadsheets/d/16lMSx3sf811uDQxNP73yeEokRLn6Deyz8_5T84-Zf5E/edit?usp=sharing.
4. Anthropic (2025). *What's new in Claude 4.6*. [online] Claude API Docs. Available at: https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6.
5. Artifex Software Inc. (2015a). *PyMuPDF 1.25.5 documentation*. [online] Readthedocs.io. Available at: https://pymupdf.readthedocs.io/en/latest/index.html.
6. Artifex Software Inc. (2015b). *PyMuPDF4LLM - PyMuPDF 1.24.9 documentation*. [online] Readthedocs.io. Available at: https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/index.html.
7. Caballar, R.D. (2023). *What is synthetic data?* [online] Ibm.com. Available at: https://www.ibm.com/think/topics/synthetic-data.
8. Claude (2023). *Tool use with Claude*. [online] Claude API Docs. Available at: https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview.
9. Claude (2026a). *Source Documentation in InstructLab*. [online] Claude. Available at: https://claude.ai/chat/ed4ebd25-4cb0-4ec8-8447-c3b2e7c7e3c8.
10. Claude (2026b). *Tool Runner (SDK)*. [online] Claude API Docs. Available at: https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-runner.
11. Claude AI (2025). *Claude AI Generated Questions Dataset*. [online] Claude.ai. Available at: https://claude.ai/share/ea9f40e3-f597-4b2b-89e9-11df7cd146bb.
12. Creative Commons (2018). *Creative Commons — Attribution 4.0 International — CC BY 4.0*. [online] Creativecommons.org. Available at: https://creativecommons.org/licenses/by/4.0/legalcode.

13. D&D Beyond Forum (2025). *An AI trained on Dungeons & Dragons - General Discussion - D&D Beyond General - D&D Beyond Forums - D&D Beyond*. [online] Dndbeyond.com. Available at: https://www.dndbeyond.com/forums/d-d-beyond-general/general-discussion/214860-an-ai-trained-on-dungeons-dragons?srsltid=AfmBOopJWF2e6tQWKRJbjMbkxvhCCh9z0YJrSiMGajEftkand7hjxkRV Post #4 by Duke_of_Vicenza.
14. Doern, C. (2025). *InstructLab, How do I use this thing? – InstructLab*. [online] Instructlab.ai. Available at: https://blog.instructlab.ai/2024/11/instructlab-how-do-i-use-this-thing/.
15. Dr Ana Rojo-Echeburúa (2025). *Building a RAG System with LangChain and FastAPI: From Development to Production*. [online] Datacamp.com. Available at: https://www.datacamp.com/tutorial/building-a-rag-system-with-langchain-and-fastapi.
16. Dr. Ashish Bamania (2025). *Build A Vector Database From Scratch To Understand RAG In Depth*. [online] Intoai.pub. Available at: https://www.intoai.pub/p/build-a-vector-database-from-scratch.
17. Facebook Research (2023). *Faiss*. [online] GitHub. Available at: https://github.com/facebookresearch/faiss.
18. Facebook Research (2025). *Home - Faiss*. [online] GitHub. Available at: https://github.com/facebookresearch/faiss/wiki.
19. GeeksforGeeks (2017). *NumPy Introduction*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/introduction-to-numpy/.
20. GeeksforGeeks (2018). *Introduction to Matplotlib*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-introduction-matplotlib/.
21. GeeksforGeeks (2020). *Functools Module in Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/functools-module-in-python/.
22. GeeksforGeeks (2024). *How to Convert JSON to Excel in Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/how-to-convert-json-to-excel-in-python/.
23. GeeksforGeeks (2025). *Sentence Transformer*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/nlp/sentence-transformer/.
24. Gemini (2026a). *API Rate Limit Troubleshooting Guide*. [online] Gemini. Available at: https://gemini.google.com/app/e44b96e61a72f84c?hl=en_GB.
25. Gemini (2026b). *InstructLab Docling Version Mismatch Fix*. [online] Gemini. Available at: https://gemini.google.com/share/f42db5ba0115.
26. GNU.org (2018). *What is Bash? (Bash Reference Manual)*. [online] Gnu.org. Available at: https://www.gnu.org/software/bash/manual/html_node/What-is-Bash_003f.html.

27. Google (2025a). *Gemma 3 model overview*. [online] Google AI for Developers. Available at: https://ai.google.dev/gemma/docs/core.
28. Google (2025b). *NotebookLM*. [online] notebooklm.google. Available at: https://notebooklm.google/.
29. Google (2026a). *Gemini 3.1 Pro Preview*. [online] Google AI for Developers. Available at: https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview.
30. Google (2026b). *Gemini API Docs and Reference | Google AI for Developers*. [online] Google for Developers. Available at: https://ai.google.dev/gemini-api/docs.
31. Google Gemini (2025). *Gemini AI Generated Questions Dataset*. [online] Gemini. Available at: https://gemini.google.com/share/63011956a2db.
32. Hugging Face (n.d.). *sentence-transformers/all-MiniLM-L6-v2 · Hugging Face*. [online] huggingface.co. Available at: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.
33. Hugging Face (n.d.). *Transformers - Quickstart*. [online] huggingface.co. Available at: https://huggingface.co/docs/transformers/quicktour.
34. huggingface (2025). *GitHub - huggingface/sentence-transformers: State-of-the-Art Text Embeddings*. [online] GitHub. Available at: https://github.com/huggingface/sentence-transformers.
35. Idan Novogroder (2024). *RAG Pipeline: Example, Tools & How to Build It*. [online] lakeFS. Available at: https://lakefs.io/blog/what-is-rag-pipeline/.
36. instructlab (2025a). *docs.instructlab.ai/docs/taxonomy/index.md at main · instructlab/docs.instructlab.ai*. [online] GitHub. Available at: https://github.com/instructlab/docs.instructlab.ai/blob/main/docs/taxonomy/index.md.
37. instructlab (2025b). *GitHub - instructlab/docs.instructlab.ai: The docs.instructlab.ai public documentation repository. PRs accepted and encouraged!* [online] GitHub. Available at: https://github.com/instructlab/docs.instructlab.ai/tree/main.
38. instructlab (2025c). *taxonomy/docs/template_qna.yaml at main · instructlab/taxonomy*. [online] GitHub. Available at: https://github.com/instructlab/taxonomy/blob/main/docs/template_qna.yaml.
39. InstructLab Authors (2024). *ilab command line reference - InstructLab documentation*. [online] Readthedocs.io. Available at: https://instructlab.readthedocs.io/v0.18.0b1/ilab.html.
40. InstructLab, Chen, C., Asghar, J. and Santamaria, L. (2025). *Welcome to InstructLab! - docs.instructlab.ai*. [online] Instructlab.ai. Available at: https://docs.instructlab.ai/.

41. Ip, J. (2025). *Leveraging LLM-as-a-Judge for Automated and Scalable Evaluation - Confident AI*. [online] Confident-ai.com. Available at: https://www.confident-ai.com/blog/why-llm-as-a-judge-is-the-best-llm-evaluation-method.
42. Ip, J. (2026). *LLM Evaluation Metrics: Everything You Need for LLM Evaluation - Confident AI*. [online] www.confident-ai.com. Available at: https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation.
43. Jani, J. (2025). *SRD v5.1*. [online] D&D Beyond. Available at: https://www.dndbeyond.com/srd.
44. Kim, D. (2025). *How to Evaluate LLMs: A Comprehensive Guide for Developers*. [online] Medium. Available at: https://medium.com/@kimdoil1211/how-to-evaluate-llms-a-comprehensive-guide-for-developers-83091ee3abc8.
45. Kumar, V. (2025). *Context Stuffing*. [online] Medium. Available at: https://medium.com/@vipulkc/context-stuffing-e904b8aa5264.
46. Label Studio (2025). *AI Model Evaluation Guide | Label Studio*. [online] Label Studio. Available at: https://labelstud.io/learningcenter/the-complete-guide-to-evaluations-in-ai/.
47. LangChain (2022). *LangChain*. [online] GitHub. Available at: https://github.com/langchain-ai/langchain.
48. LangChain Docs (2025). *Split markdown - text splitter integration*. [online] Langchain.com. Available at: https://docs.langchain.com/oss/python/integrations/splitters/markdown_header_metadata_splitter.
49. LangChain Reference (2026a). *MarkdownHeaderTextSplitter*. [online] Langchain.com. Available at: https://reference.langchain.com/python/langchain-text-splitters/markdown/MarkdownHeaderTextSplitter.
50. LangChain Reference (2026b). *RecursiveCharacterTextSplitter*. [online] Langchain.com. Available at: https://reference.langchain.com/python/langchain-text-splitters/character/RecursiveCharacterTextSplitter.
51. Langfuse (2022). *LLM-as-a-Judge Evaluation - Langfuse*. [online] Langfuse.com. Available at: https://langfuse.com/docs/evaluation/evaluation-methods/llm-as-a-judge.

52. Levy, M. (2026). *What It Takes to Build a RAG System from Scratch in Python*. [online] Dataquest. Available at: https://www.dataquest.io/blog/build-a-rag-system-from-scratch/.
53. MatplotLib (2024a). *Gallery — Matplotlib 3.4.2 documentation*. [online] matplotlib.org. Available at: https://matplotlib.org/stable/gallery/index.html.
54. MatplotLib (2024b). *Introduction to Axes (or Subplots) — Matplotlib 3.9.2 documentation*. [online] Matplotlib.org. Available at: https://matplotlib.org/stable/users/explain/axes/axes_intro.html.
55. MatplotLib (2024c). *Quick start guide — Matplotlib 3.8.0 documentation*. [online] matplotlib.org. Available at: https://matplotlib.org/stable/users/explain/quick_start.html.
56. Matt Cone and contributors (n.d.). *Markdown Guide*. [online] www.markdownguide.org. Available at: https://www.markdownguide.org/.
57. Meta (2022). *Llama 3*. [online] Meta Llama. Available at: https://www.llama.com/models/llama-3/.
58. Milvus (2026a). *How can I implement temperature and max tokens in OpenAI's API?* [online] Milvus.io. Available at: https://milvus.io/ai-quick-reference/how-can-i-implement-temperature-and-max-tokens-in-openais-api.
59. Milvus (2026b). *What is the simplest way to encode a list of sentences into embeddings using a pre-trained Sentence Transformer model?* [online] Milvus.io. Available at: https://milvus.io/ai-quick-reference/what-is-the-simplest-way-to-encode-a-list-of-sentences-into-embeddings-using-a-pretrained-sentence-transformer-model.
60. Mistral AI (2025). *Introducing Mistral 3*. [online] Mistral.ai. Available at: https://mistral.ai/news/mistral-3.
61. Moore, J. (2025). *RAG vs. Prompt Stuffing: Overcoming Context Window Limits for Large, Information-Dense Documents*. [online] Spyglassmtg.com. Available at: https://www.spyglassmtg.com/blog/rag-vs.-prompt-stuffing-overcoming-context-window-limits-for-large-information-dense-documents.
62. Morgan, A. (2024). *BERTScore For LLM Evaluation*. [online] Comet. Available at: https://www.comet.com/site/blog/bertscore-for-llm-evaluation/.
63. NG, P. (2025). *Creating a Vector Database for RAG*. [online] Medium. Available at: https://praveenng.medium.com/creating-a-vector-database-for-rag-471aca771bce.
64. Ong, R. (2024). *What Is Faiss (Facebook AI Similarity Search)?* [online] Datacamp.com. Available at:

https://www.datacamp.com/blog/faiss-facebook-ai-similarity-search?dc_referrer=https%3A%2F%2Fwww.google.com%2F.

65. OpenAI (2024). *Developer quickstart | OpenAI API*. [online] Openai.com. Available at: https://developers.openai.com/api/docs/quickstart?lang=python&language=python.
66. OpenAI (2025a). *Best Practices for API Key Safety | OpenAI Help Center*. [online] help.openai.com. Available at: https://help.openai.com/en/articles/5112595-best-practices-for-api-key-safety.
67. OpenAI (2025b). *GPT-5.4 Model | OpenAI API*. [online] Openai.com. Available at: https://developers.openai.com/api/docs/models/gpt-5.4.
68. OpenAI (2025c). *Models | OpenAI API*. [online] Openai.com. Available at: https://developers.openai.com/api/docs/models.
69. OpinionatedHeron (2026). *GitHub - OpinionatedHeron/RAW_DnD_Project: Project to investigate the use of Large Language Models and Retrieval-Augmented Generation frameworks as a rule-assistance tool for Tabletop Role-Playing Games (TTRPGs) with a specific focus on Dungeons and Dragons 5th Edition. Test based development and investigation.* [online] GitHub. Available at: https://github.com/OpinionatedHeron/RAW_DnD_Project.
70. oxylabs (2025). *GitHub - oxylabs/python-cache-tutorial: A guide to caching web scraping scripts in Python.* [online] GitHub. Available at: https://github.com/oxylabs/python-cache-tutorial.
71. Perplexity (2025). *What is Perplexity? | Perplexity Help Center*. [online] Perplexity.ai. Available at: https://www.perplexity.ai/help-center/en/articles/10352155-what-is-perplexity.
72. Perplexity (2026). *Perplexity*. [online] Perplexity AI. Available at: https://www.perplexity.ai/search/i-am-looking-to-investigate-us-CQ7kDYmBTKqNQpyAEefO3Q#5.
73. Pierre, S. (2022). *Function Wrappers in Python: Model Runtime and Debugging | Towards Data Science*. [online] Towards Data Science. Available at: https://towardsdatascience.com/function-wrappers-in-python-model-runtime-and-debugging-243da483b9d6/.
74. Pilkhan, M. (2024). *Building a RAG pipeline — Step-By-Step*. [online] Medium. Available at: https://medium.com/@mahakal001/building-a-rag-pipeline-step-by-step-0a5e1ac68562.
75. Python Coding (2024). *PDF Manipulation using Python — fitz Library*. [online] Medium. Available at: https://pythonclcoding.medium.com/pdf-manipulation-using-python-fitz-library-619227d59f3d.

76. Python Software Foundation (2019a). *pickle — Python object serialization*. [online] Python.org. Available at: https://docs.python.org/3/library/pickle.html.
77. Python Software Foundation (2019b). *venv — Creation of virtual environments — Python 3.8.1 documentation*. [online] Python.org. Available at: https://docs.python.org/3/library/venv.html.
78. Real Python (2025). *functools*. [online] Realpython.com. Available at: https://realpython.com/ref/stdlib/functools/.
79. Red Hat (2023). *What is YAML?* [online] www.redhat.com. Available at: https://www.redhat.com/en/topics/automation/what-is-yaml.
80. Red Hat (2025). *What is InstructLab?* [online] Redhat.com. Available at: https://www.redhat.com/en/topics/ai/what-is-instructlab.
81. Sbert.net (2025). *SentenceTransformer — Sentence Transformers Documentation*. [online] Sbert.net. Available at: https://sbert.net/docs/package_reference/sentence_transformer/SentenceTransformer.html .
82. Sbert.net (2026). *Quickstart — Sentence Transformers Documentation*. [online] Sbert.net. Available at: https://sbert.net/docs/quickstart.html#sparse-encoder.
83. Shin, M. (2025). *RAG indexing: Structure and evaluate for grounded LLM answers*. [online] Meilisearch.com. Available at: https://www.meilisearch.com/blog/rag-indexing.
84. Sinha, D. (2025). *The Ultimate Guide to Chunking Strategies for RAG Applications with Databricks*. [online] Databricks.com. Available at: https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications/ba-p/113089.
85. Stohn, C. and Enciso, M. (2025). *How to Evaluate Generative AI Output Effectively | Clarivate*. [online] Clarivate. Available at: https://clarivate.com/academia-government/blog/evaluating-the-quality-of-generative-ai-output-methods-metrics-and-best-practices/.
86. The Markdown Guide (n.d.). *Extended Syntax | Markdown Guide*. [online] www.markdownguide.org. Available at: https://www.markdownguide.org/extended-syntax/.
87. Tiiiger (2023). *GitHub - Tiiiger/bert_score: BERT score for text generation*. [online] GitHub. Available at: https://github.com/Tiiiger/bert_score/tree/master.
88. Tiiiger (2025). *BERTScore Compute Image*. [online] GitHub. Available at: https://github.com/Tiiiger/bert_score/blob/master/bert_score.png.
89. W3Schools (2022). *Introduction to NumPy*. [online] www.w3schools.com. Available at: https://www.w3schools.com/python/numpy/numpy_intro.asp.

90. w3schools (2025). *W3Schools.com*. [online] W3schools.com. Available at: https://www.w3schools.com/python/python_virtualenv.asp.
91. Walied, H. (2025). *Building a RAG from Scratch: A Beginner's Guide (Part 1: The Basic Pipeline)*. [online] DEV Community. Available at: https://dev.to/hadywalied/building-a-rag-from-scratch-a-beginners-guide-part-1-the-basic-pipeline-17b0.
92. Wizards of the Coast (2016a). *System Reference Document 5.1*. [online] Wizards of the Coast. Available at: https://media.wizards.com/2016/downloads/DND/SRD-OGL_V5.1.pdf.
93. Wizards of the Coast (2016b). *System Reference Document 5.1 - CCBY04 License*. [online] D&D Beyond. Available at: https://www.dndbeyond.com/attachments/39j2li89/SRD5.1-CCBY4.0License.pdf.
94. Wolff, H. (2023). *RAG 101: Demystifying Retrieval-Augmented Generation Pipelines*. [online] NVIDIA Technical Blog. Available at: https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/.
95. X. McKie, J. and Artifex Software, Inc. (n.d.). *GitHub - pymupdf/PyMuPDF: Python bindings for MuPDF's rendering library*. [online] GitHub. Available at: https://github.com/pymupdf/PyMuPDF.
96. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q. and Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. *arXiv:1904.09675 [cs]*. [online] doi:https://doi.org/1904.09675.